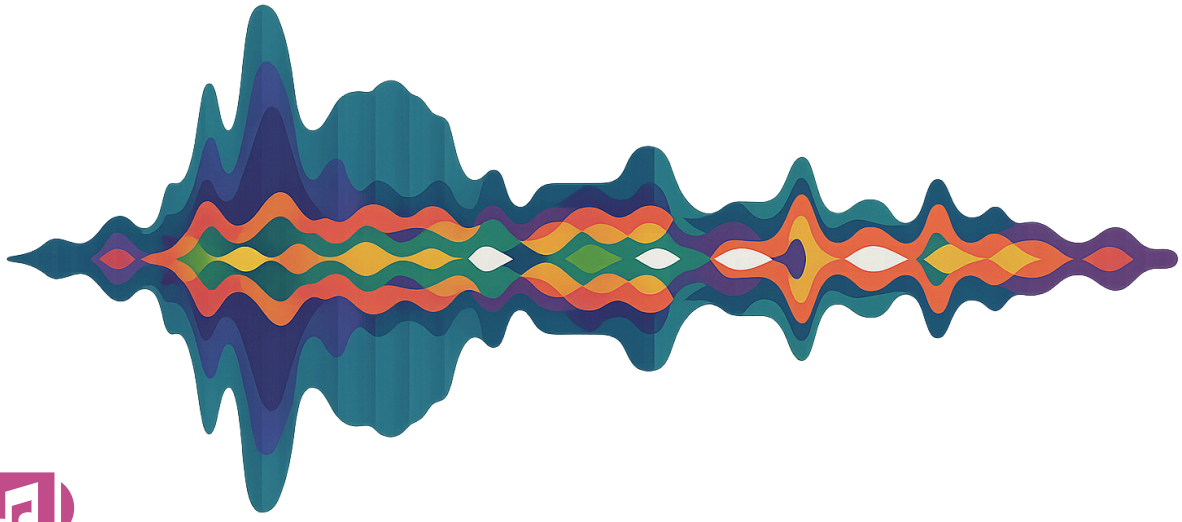


FICHA TÉCNICA_ PROJETO 2 HIPÓTESES

Equipe: ***Maria Eduarda Carvalho e Wellen Mariah Lisboa .***

Jornada de Dados Laboratória

Contexto:

Uma gravadora levantou uma série de hipóteses sobre o que faz uma música ser mais ouvida. Essas hipóteses incluem:

- Músicas com BPM (Batidas Por Minuto) mais altos fazem mais sucesso em termos de número de streams no Spotify.
- As músicas mais populares no ranking do Spotify também possuem um comportamento semelhante em outras plataformas, como a Deezer.
- A presença de uma música em um maior número de playlists está correlacionada com um maior número de streams.
- Artistas com um maior número de músicas no Spotify têm mais streams.
- As características da música influenciam o sucesso em termos de número de streams no Spotify.

O objetivo é saber quais fatores contribuem para o sucesso de uma música em diversas plataformas de streaming. Com o intuito de lançar uma novo artista no mercado musical, a gravadora em questão deseja receber recomendações estratégicas com base em dados para que decisões informadas possam ser tomadas e suas chances de alcançar o “sucesso” aumentem.

Objetivos:

- Analisar e responder as hipóteses levantadas pela gravadora.
- Identificar padrões ou características que possam influenciar a popularidade de uma música em plataformas como o Spotify, AppleMusic e Deezer, determinando os fatores que contribuem para o sucesso de uma música.

Ferramentas e Tecnologias:

BigQuery (utilizando SQL);
PowerBI (foi utilizado Python);
Google Colab.
Outras tecnologias:
Notion (gerenciamento de tempo e progresso);
Google Documentos (montagem da ficha técnica).

▼ Processamento e Análises

▼ 1. Importação dos Dados:

a. A ferramenta utilizada nesse projeto é o BigQuery. Para importar o dataset recebido através de um arquivo compactado foi necessário, primeiramente descompactar o arquivo, em seguida criar um projeto no console do BigQuery (***projeto2lab***), logo após criar um conjunto de dados (***projetohipoteses_lab2***) e inserir na parte “Adicionar Dados Próprios” de forma manual, cada arquivo CSV.

b. Houve um erro ao importar uma das tabelas, mas especificamente a planilha referente aos dados do Spotify. Para solucionar esse erro, abrimos a tabela no bloco de Notas para retirar o “()” do nome da coluna *artists_name*.

▼ 2. Limpeza dos Dados:

a. Identificar espaços vazios (nulos):

```
SELECT
 COUNT (*)
 FROM `projeto2lab-457400.projetohipoteses_lab2.tracktechnicalinfo`
WHERE track_id IS NULL
```

```
SELECT
COUNT (*)
FROM `projeto2lab.projetohipoteses_lab2.tracktechnicalinfo`
WHERE key IS NULL;
```

O mesmo processo foi repetido em todas as variáveis de cada tabela, sendo encontrados valores vazios apenas em duas planilhas (*trackincompetition* e *tracktechnicalinfo)*:

- 50 (cinquenta) valores nulos na variável i*n_shazam_charts* da tabela *trackincompetition*;
- 95 (noventa e cinco) valores nulos na variável *key* da tabela *tracktechnicalinfo*;
- A tabela *trackinspotify* teve um total de 0 valores nulos.

Na tabela *tracktechnicalinfo*, optamos por preencher os valores ausentes da coluna key com a indicação de 'Não Informado', considerando que a variável possui o tipo de dado string. Já em relação aos valores nulos da tabela *trackincompetition*, a coluna *in_shazam_charts* foi mantida em seu formato original, uma vez que, para fins analíticos, consideramos os espaços vazios como representações válidas da presença na lista.

---

b. Verificar valores duplicados:

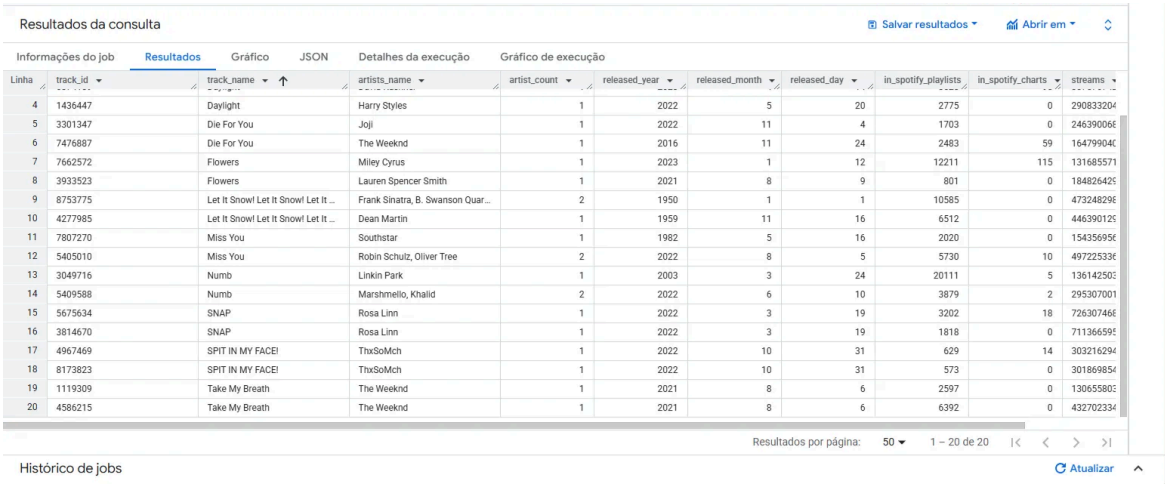
```
SELECT track_name, artist_s__name,
COUNT (*) FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
GROUP BY track_name, artist_s__name
HAVING COUNT (*) > 1
```

Nessa consulta, selecionamos as variáveis *track_name* e *artist_s__name* para verificar repetições. O GROUP BY agrupa os registros com base em valores únicos dessas duas colunas, enquanto o COUNT(*) retorna o número de vezes que cada combinação aparece. Em seguida, o HAVING COUNT(*) > 1 filtra apenas os casos

em que a mesma combinação aparece mais de uma vez, ou seja, os registros duplicados.

Posteriormente, uma nova consulta incluindo também a coluna *track_id* foi realizada, com o objetivo de verificar se havia repetições considerando essas três variáveis em conjunto. No entanto, o resultado retornado foi zero, indicando que não existem duplicações quando o *track_id* é incluído na análise. Ou seja, os track_ids são únicos mesmo entre registros com nomes de músicas e artistas repetidos.

O mesmo processo foi repetido nas demais tabelas mas apenas foram identificados 4 (quatro) registros duplicados na tabela *trackinspotifylike.*



Linha	track_id	track_name	artists_name	artist_count	released_year	released_month	released_day	in_spotify_playlists	in_spotify_charts	streams
4	1436447	Daylight	Harry Styles	1	2022	5	20	2775	0	290833204
5	3301347	Die For You	Joji	1	2022	11	4	1703	0	246390068
6	7476887	Die For You	The Weeknd	1	2016	11	24	2483	59	164799040
7	7662572	Flowers	Miley Cyrus	1	2023	1	12	12211	115	131685571
8	3933523	Flowers	Lauren Spencer Smith	1	2021	8	9	801	0	184826429
9	8753775	Let It Snow! Let It Snow! Let It ...	Frank Sinatra, B. Swanson Quar...	2	1950	1	1	10585	0	473248296
10	4277985	Let It Snow! Let It Snow! Let It ...	Dean Martin	1	1959	11	16	6512	0	446390129
11	7807270	Miss You	Southstar	1	1982	5	16	2020	0	154356956
12	5405010	Miss You	Robin Schulz, Oliver Tree	2	2022	8	5	5730	10	497225336
13	3049716	Numb	Linkin Park	1	2003	3	24	20111	5	136142503
14	5409588	Numb	Marshmello, Khalid	2	2022	6	10	3879	2	295307001
15	5675634	SNAP	Rosa Linn	1	2022	3	19	3202	18	726307466
16	3814670	SNAP	Rosa Linn	1	2022	3	19	1818	0	711366595
17	4967469	SPIT IN MY FACE!	ThxSoMch	1	2022	10	31	629	14	303216294
18	8173823	SPIT IN MY FACE!	ThxSoMch	1	2022	10	31	573	0	301869854
19	1119309	Take My Breath	The Weeknd	1	2021	8	6	2597	0	130655803
20	4586215	Take My Breath	The Weeknd	1	2021	8	6	6392	0	432702334

Para analisar o comportamento das duplicadas em relação às demais variáveis da tabela *trackinspotifylike*, foi executado a seguinte consulta:

```
WITH duplicados AS (
  SELECT
   track_name,
   artist_s__name
  FROM
    `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
  GROUP BY
    track_name,
    artist_s__name
  HAVING
    COUNT(*) > 1
)
SELECT
  t.*
```

```
FROM
  `projeto2lab.projetohipoteses_lab2.trackinspotifylike` t
JOIN
  duplicados d
ON
  t.track_name = d.track_name
  AND t.artist_s__name = d.artist_s__name
```

Foi feita uma consulta com a cláusula WITH para criar uma mini tabela temporária, onde essa subconsulta identificou as combinações que apareciam mais de uma vez na tabela original, ou seja, as duplicadas.

Com isso, foi possível realizar uma segunda consulta, que retornou todas as variáveis que correspondiam às duplicadas (identificadas pelas colunas *track_name* e *artist_s__name*) e por meio de um JOIN entre a tabela original e a tabela temporária (onde as demais linhas e colunas originais correspondem às colunas temporárias), foi estabelecido uma correlação entre elas, permitindo uma análise mais detalhada dessas entradas.

Por fim, tomamos a decisão de remover as linhas duplicadas com base no critério de que as faixas com o track_id estivesse em algum ponto disconformes, seriam removidas. Como o exemplo da faixa “About Damn Time” da cantora-compositora e rapper norte-americana, Lizzo, que possuía data e mês de lançamento diferentes. Nos outros 3 (três) casos de duplicatas, o critério de remoção utilizado foi a baixa quantidade de streams e números em playlists.

A consulta utilizada para fazer esse processo foi:

```
SELECT *
FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
WHERE track_id NOT IN ('5080031', '3814670', '1119309', '817382
3');
```

O NOT IN foi aplicado de forma que ele retornasse todos os resultados menos os listados, trazendo apenas 949 resultados dos 953 originais.



Linha	track_id	track_name	artist_s__name	artist_count	released_year	released_month	released_day	in_spotify
1	1406287	I Wanna Be Yours	Arctic Monkeys	1	2013	1	1	
2	3921634	Blank Space	Taylor Swift	1	2014	1	1	
3	4796316	Style	Taylor Swift	1	2014	1	1	
4	4527090	Yellow	Chris Molitor	1	1999	1	1	
5	3126367	Viva La Vida	Coldplay	1	2008	1	1	
6	8502696	Riptide	Vance Joy	1	1975	1	1	
7	1216337	Summertime Sadness	Lana Del Rey	1	2011	1	1	
8	1729982	Lover	Taylor Swift	1	2012	1	1	
9	2394886	Mockingbird	Eminem	1	2004	1	1	
10	4350729	Set Fire to the Rain	Adele	1	2011	1	1	
11	7811952	Money Trees	Kendrick Lamar, Jay Rock	2	2012	1	1	
12	4458161	Radio	Lana Del Rey	1	2011	1	1	
13	3253565	Perfect	Ed Sheeran	1	2017	1	1	

c. Identificar e tratar dados fora do escopo de análise

Decidimos não remover nenhuma coluna de nenhuma tabela trabalhada com base no critério de que até aquele momento da análise, todas as variáveis eram importantes. Mas para testar a usabilidade foi usada a formula:

```
SELECT *
FROM `projeto2lab-457400.projetohipoteses_lab2.trackinspotify`
WHERE NOT track_id = '5080031' AND track_name NOT IN ('SNAP',
'SPIT IN MY FACE!', 'Take My Breath')
```

d. Identificar e tratar dados discrepantes em variáveis categóricas.

OBS.: Como havíamos retirado o "()" da tabela *trackinspotify* pelo bloco de notas, ao fazer o upload do dataset no console, ele removeu todos tipos de caracteres que não fossem alfanuméricos. Portanto, tivemos que "subir" uma nova tabela intitulada *trackinspotifylike* para fazer o tratamento dos de forma correta.

Sendo assim, ao realizar uma análise visual rápida pela tabela *trackinspotifylike,* percebemos a presença de caracteres especiais de forma evidente em algumas colunas. Para localizar essas anomalias, foi realizada uma consulta utilizando a cláusula LIKE para encontrar valores que começavam com a letra B na coluna streams, onde estava visível a irregularidade da coluna, já que ela originalmente é do tipo integer.

```
SELECT *
FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
```

```
WHERE streams LIKE 'B%'
```

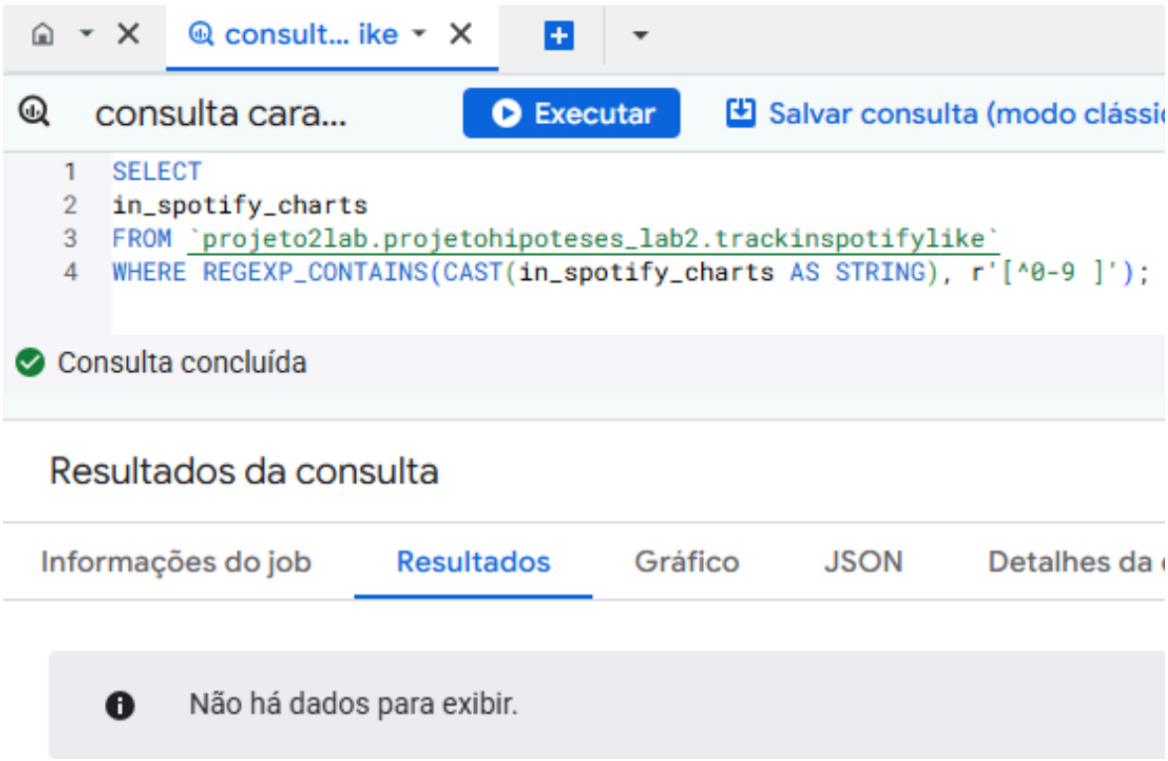
Essa consulta nos retornou o resultado esperado, mas como já mencionado, outras variáveis também apresentaram caracteres especiais visíveis, o que exigiu um aprofundamento das consultas.

Por isso, decidimos utilizar a função REGEXP_CONTAINS para identificar qualquer tipo de caractere especial nas variáveis da tabela. A expressão regular utilizada foi: r'[^a-zA-Z0-9]', que busca por qualquer caractere que não seja alfanumérico.

Em alguns casos, como em variáveis numéricas, foi necessário usar o CAST para converter os valores em STRING, já que o REGEXP_CONTAINS só funciona com dados do tipo string. Um exemplo disso foi a variável *in_spotify_charts*, que originalmente era do tipo integer. A consulta a seguir foi utilizada para identificar a presença de caracteres especiais na variável após a conversão para string:

```
SELECT
in_spotify_charts
FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
WHERE REGEXP_CONTAINS(CAST(in_spotify_charts AS STRING), r'[^0-
9 ]');
```

Nessa consulta, a função CAST foi usada para transformar a variável *in_spotify_charts* em STRING, permitindo que o REGEXP_CONTAINS fosse aplicado corretamente. A expressão regular foi reduzida para r'[^0-9]', onde ela busca por qualquer caractere que não seja numérico ou espaço, ajudando a identificar valores discrepantes.



Esse mesmo processo se repetiu nas demais colunas e tabelas. Utilizamos o REGEXP_CONTAINS para identificar quais caracteres diferentes dos alfanuméricos estavam presentes nas variáveis, e o LIKE para buscar exatamente o caractere em questão.

Com isso, verificamos que o caractere especial predominante era o chamado “caractere de substituição”, utilizamos a cláusula LIKE para retornar apenas as linhas que continham esse caractere específico.

```
2  artist_s__name
3  FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
4  WHERE artist_s__name LIKE r'%�%';
```

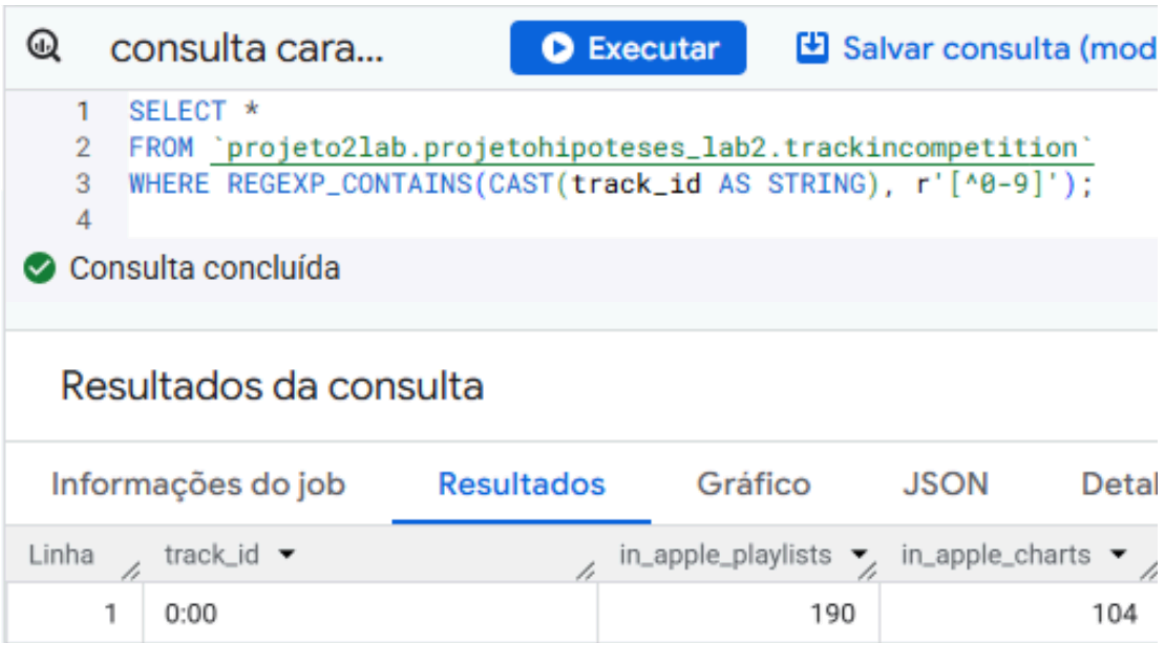
Consulta concluída

Resultados da consulta

Informações do job | Resultados | Gráfico | JSON | D

Linha	artist_s__name
1	Ti��
2	Elley Duh�
3	Quevedo, La Pantera, Juseph, Cruz Cafun��, B��jo, Abhir Hathi
4	Mar��lia Mendon��a, Hugo...

Nas demais tabelas, *trackincompetition* e *tracktechnicolinfo*, apenas um resultado diferente foi apresentado. Os caracteres de substituição não se apresentavam nas variáveis dessas tabelas.



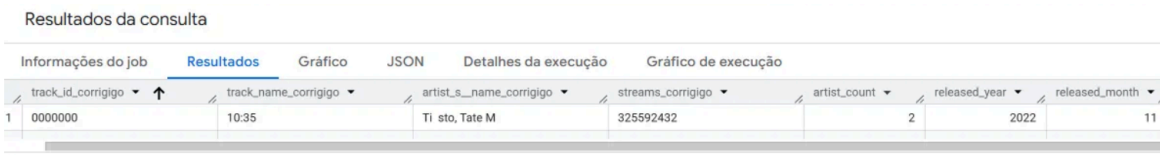
Ao fim do processo de verificação, decidimos ajustar essas anomalias na tabela *trackinspotifylike*, pois ela era a que apresentava a maior quantidade de caracteres especiais. Para isso, foi utilizada a seguinte consulta para substituir as irregularidades em variáveis específicas e, ao mesmo tempo, manter as demais variáveis originais, que não sofreram alterações:

```
SELECT
  REGEXP_REPLACE(track_id, r'0:00', '0000000') AS track_id_corr
igido,
  REGEXP_REPLACE(track_name, r'', ' ') AS track_name_corrigid
o,
  REGEXP_REPLACE(artist_s__name, r'', ' ') AS artist_s__name_c
orrigido,
  REGEXP_REPLACE(streams, r'BPM110KeyAModeMajorDanceability53Va
lence75Energy69Acousticness7Instrumentalness0Liveness17Speechin
ess3', '0') AS streams_corrigido,


  artist_count,
  released_year,
  released_month,
  released_day,
  in_spotify_playlists,
  in_spotify_charts,
```

```
FROM
  `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
```

Após o tratamento desses dados, o resultado que obtivemos foi o seguinte:



O mesmo processo foi realizado nas demais tabelas, porém nelas, a mudança ocorreu apenas na variável track_id, que outrora possuía uma das informações como “0:00”.

e. Identificar e tratar dados discrepantes em variáveis numéricas

```
1  SELECT
2  MAX(`speechiness_%`) as maximo,
3  MIN(`speechiness_%`) AS minimo,
4  AVG(`speechiness_%`) AS media,
5  SUM(`speechiness_%`) AS soma
6    FROM `projeto2lab.projetohipoteses_lab2.tracktechnicalinfo`
```

Consulta concluída

Resultados da consulta

Informações do job | Resultados | Gráfico | JSON | Detalhes da execução

Linha	maximo	minimo	media	soma
1	64	2	10.13116474291...	9655

Em alguns casos, onde as variáveis apresentavam dados tipo string, utilizamos a cláusula DISTINCT para verificar a regularidade e a qualidade das informações. Não foram identificadas anomalias ou discrepância relevantes, considerando que valores com grandezas elevadas eram esperados, de acordo com o contexto do projeto.

Sendo assim, nenhuma informação foi retirada nesta meta.

▼ 3. Verificar e alterar tipo de dados:

Mantivemos o processo habitual: analisamos todas as variáveis presentes em cada uma das tabelas e constatamos que, em sua maioria, os dados estavam adequados. Contudo, identificamos a necessidade de alterar o tipo da variável *streams*, presente na tabela *trackinspotifylike*, que originalmente era do tipo string devido a presença de caracteres alfanuméricos. Convertê-la para o para o tipo INTEGER foi fundamental para possibilitar a correta manipulação numérica dos valores, resultando na transformação apropriada da coluna.

Antes da última etapa desta meta, decidimos criar uma versão limpa de cada tabela, utilizando **VIEWs**, para facilitar a unificação em um único dataset, proposto na etapa seguinte. Dessa forma, todos os processos descritos anteriormente foram consolidados em um único código, ajustado conforme as particularidades de cada tabela.

Além disso, antes de iniciar essa consolidação, optamos por eliminar a linha que continha valores nulos na coluna de streams na tabela *trackinspotifylike*, a fim de evitar conflitos futuros com a plataformas como o Power BI. Assim, o track_id correspondente a música foi incluído em uma cláusula NOT IN e removida juntamente com as faixas duplicadas.

Para *trackinspotifylike*, a seguinte consulta foi utilizada:

```
WITH base_convertida AS (
  SELECT *,
    DATE(released_year, released_month, released_day) AS data_l
acamento
  FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike`
)
SELECT
REGEXP_REPLACE(track_id, r'0:00', '0000000') AS track_id,
REGEXP_REPLACE(track_name, r'', ' ') AS track_name,
REGEXP_REPLACE(artist_s__name, r'', ' ') AS artist_s__name,
  artist_count,
  in_spotify_playlists,
  in_spotify_charts,
  data_lacamento,
  streams
FROM base_convertida
```

```
WHERE track_id NOT IN ('5080031', '3814670', '1119309', '817382
3', '4061483')
```

Para *trackincompetition*, a seguinte consulta foi utilizada:

```
SELECT
    REGEXP_REPLACE(track_id, r'0:00', '0000000') AS track_id,
    in_apple_playlists,
    in_apple_charts,
    in_deezer_playlists,
    in_deezer_charts,
    in_shazam_charts,
  FROM `projeto2lab.projetohipoteses_lab2.trackincompetition`
WHERE track_id NOT IN ('5080031', '3814670', '1119309', '817382
3', '4061483')
```

Para *trackitecnicalinfo*, a seguinte consulta foi utilizada:

```
SELECT
     REGEXP_REPLACE(track_id, r'0:00', '0000000') AS track_id,
    bpm,
    IFNULL(key, 'Não Informado') AS key,
    mode,
    `danceability_%`,
    `valence_%`,
    `energy_%`,
    `acousticness_%`,
    `instrumentalness_%`,
    `liveness_%`,
    `speechiness_%`
  FROM `projeto2lab.projetohipoteses_lab2.tracktechnicalinfo`
WHERE track_id NOT IN ('5080031', '3814670', '1119309', '817382
3', '4061483')
```

▼ Criar novas variáveis + Unir Tabelas + Construir Tabela Auxiliares:

A **criação de novas variáveis** e a **unificação das tabelas** em um único dataset foram feitas de forma paralela. Conforme surgiam novas demandas analíticas, essa

consulta foi continuamente atualizada ao longo do processo de exploração dos dados e construção de gráficos.

Todos os procedimentos aprendidos nas etapas anteriores foram aplicados aqui, incluindo a criação de tabelas temporárias com o WITH, agregações utilizando GROUP BY e HAVING, cálculos de medidas de tendência central, como AVG, MEDIAN e SUM e junção de tabelas JOIN. Além disso, utilizamos cláusulas CASE para criar expressões condicionais que converteram duas variáveis originalmente do tipo string em representações numéricas, conforme as necessidades da análise.

Tabela unificada versão final:

```
WITH tracks_charts_sp AS (
  SELECT
artist_s__name,
COUNT (*) AS total_tracks_charts_artist_sp
FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
WHERE in_spotify_charts > 0
GROUP BY artist_s__name
),
tracks_charts_ap AS (
SELECT
artist_s__name,
COUNT (*) AS total_track_charts_artist_ap
FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
WHERE in_apple_charts > 0
GROUP BY artist_s__name
),
tracks_charts_dz AS (
  SELECT
  artist_s__name,
  COUNT(*) AS total_tracks_charts_artist_dz
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
  WHERE in_deezer_charts > 0
  GROUP BY artist_s__name
),
tracks_charts_sz AS (
  SELECT
  artist_s__name,
  COUNT(*) AS total_tracks_charts_artist_sz
```

```
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
  WHERE in_shazam_charts > 0
  GROUP BY artist_s__name
),
soma_playlists AS (
SELECT
track_id,
(in_spotify_playlists + in_apple_playlists + in_deezer_playlist
s) AS soma_playlists
FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
),
media_playlists AS (
  SELECT
    track_id,
    CAST(ROUND(in_spotify_playlists + in_apple_playlists + in_d
eezer_playlists / 3) AS INT64) AS media_playlists
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
),
media_charts AS (
  SELECT
    track_id,
    CAST(ROUND(SAFE_DIVIDE(in_spotify_charts + in_apple_charts
+ in_deezer_charts + IFNULL(in_shazam_charts, 0), 4)) AS INT64)
AS media_charts
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
),
quartil_streams AS (
  SELECT
    track_id,
    CAST(streams AS INT64) AS streams,
    NTILE(4) OVER (ORDER BY CAST(streams AS INT64)) AS quartil_
streams
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
  WHERE SAFE_CAST(streams AS INT64) IS NOT NULL
)
```

```sql
SELECT
  s.track_id,
  s.track_name,
  s.artist_s__name,
  s.artist_count,
  s.data_lacamento,
  s.in_spotify_charts,
  s.in_spotify_playlists,
  c.in_apple_charts,
  c.in_apple_playlists,
  c.in_deezer_charts,
  c.in_deezer_playlists,
  c.in_shazam_charts,
  CAST(s.streams AS INT64) AS streams,
  qs.quartil_streams,
CASE
  WHEN qs.quartil_streams = 4 THEN 'Alto'
  WHEN qs.quartil_streams = 3 THEN 'Médio'
  WHEN qs.quartil_streams = 2 THEN 'Baixo'
  WHEN qs.quartil_streams = 1 THEN 'Muito baixo'
END AS classificacao_streams,
  IFNULL(sp.total_tracks_charts_artist_sp, 0) AS total_tracks_c
harts_artist_sp,
  IFNULL(ap.total_track_charts_artist_ap, 0) AS total_track_cha
rts_artist_ap,
  IFNULL(dz.total_tracks_charts_artist_dz, 0) AS total_tracks_c
harts_artist_dz,
  IFNULL(sz.total_tracks_charts_artist_sz, 0) AS total_tracks_c
harts_artist_sz,
  pl.soma_playlists,
  ROUND(mp.media_playlists, 2) AS media_playlists,
  m.media_charts,
  i.bpm,
  i.key,
  CASE
      WHEN key = "Não Informado" THEN 0
      WHEN key = "C#" THEN 1
      WHEN key = "D" THEN 2
      WHEN key = "D#" THEN 3
```

```
      WHEN key = "E" THEN 4
      WHEN key = "F" THEN 5
      WHEN key = "F#" THEN 6
      WHEN key = "G" THEN 7
      WHEN key = "G#" THEN 8
      WHEN key = "A" THEN 9
      WHEN key = "A#" THEN 10
      WHEN key = "B" THEN 11
      ELSE NULL
    END AS key_inter,
  i.mode,
  CASE
  WHEN i.mode = 'Minor' THEN 1
  ELSE 0
END AS mode_inter,
  i.`danceability_%`,
  i.`valence_%`,
  i.`energy_%`,
  i.`acousticness_%`,
  i.`instrumentalness_%`,
  i.`liveness_%`,
  i.`speechiness_%`
FROM `projeto2lab.projetohipoteses_lab2.trackinspotifylike_limp
a`  AS s
JOIN `projeto2lab.projetohipoteses_lab2.trackincompetition_limp
o` AS c ON s.track_id = c.track_id
JOIN `projeto2lab.projetohipoteses_lab2.tracktechnicalinfo_limp
o` AS i ON s.track_id = i.track_id
LEFT JOIN tracks_charts_sp AS sp ON s.artist_s__name = sp.artis
t_s__name
LEFT JOIN tracks_charts_ap AS ap ON s.artist_s__name = ap.artis
t_s__name
LEFT JOIN tracks_charts_dz AS dz ON s.artist_s__name = dz.artis
t_s__name
LEFT JOIN tracks_charts_sz AS sz ON s.artist_s__name = sz.artis
t_s__name
LEFT JOIN soma_playlists AS pl ON s.track_id = pl.track_id
LEFT JOIN media_playlists AS mp ON s.track_id = mp.track_id
```

```
LEFT JOIN media_charts AS m ON s.track_id = m.track_id
LEFT JOIN quartil_streams AS qs ON s.track_id = qs.track_id
```

Por fim, uma nova tabela adicional foi criada para permitir uma melhor avaliação da hipótese 3, que questionava se a presença de uma música em um maior número de playlists estaria correlacionada com um maior volume de streams, possibilitando uma análise mais precisa dessa relação.

```
SELECT
  artist_s__name,
  COUNT(*) AS total_tracks,
  SUM(streams) AS total_streams,
FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
GROUP BY artist_s__name
ORDER BY total_tracks DESC;
```

Informações do job | Resultados | Gráfico | JSON

Linha	artist_s__name	total_tracks	total_streams
1	Taylor Swift	34	14053658300
2	The Weeknd	21	14054897067
3	SZA	19	4557811204
4	Bad Bunny	19	9997799607
5	Harry Styles	17	11608645649
6	Kendrick Lamar	12	3033135947
7	Morgan Wallen	11	1666199217
8	Ed Sheeran	9	13908947204
9	Drake, 21 Savage	8	2076567740
10	BTS	8	4389891591

Com isso, trabalhamos com cenário onde obtivemos um total de 948 músicas, 644 artistas, 887 faixas solos, 67 faixas com colaboração e músicas com anos de lançamento entre 1945 à 2023.

▼ Exploração e Análise dos Dados

▼ 1. Análise Exploratória:

Para esta etapa, incorporamos uma nova ferramenta ao processo: o Power BI, utilizada para a visualização e exploração dos dados. Com ela, conseguimos compreender as informações de forma mais direta e intuitiva, por meio da criação de gráficos e tabelas-matrizes que facilitaram a identificação de padrões e tendências.

Além disso, aprofundamos a exploração dos dados com a elaboração de gráficos de dispersão, visando analisar as correlações entre as variáveis das hipóteses. Complementamos esse processo utilizando o Python, para gerar gráficos do tipo de histograma, possibilitando uma análise mais detalhada da distribuição das variáveis numéricas.

No decorrer dessa meta, algumas novas variáveis também foram adicionadas ao BigQuery com o objetivo de facilitar e enriquecer a análise exploratória.

Na primeira etapa, carregamos o dataset na ferramenta e, durante a verificação dos dados, identificamos que a variável *streams* havia sido carregada como *string*. Isso ocorreu porque, no processo inicial de limpeza e transformação, optamos por manter essa variável em seu formato original.

Naquele momento, decidimos prosseguir com a investigação, apesar dessa inconsistência. Contudo, ao iniciarmos cálculos básicos e operações com medidas de tendência central, como média, mediana, desvio padrão e variância, percebemos que essa coluna não poderia ser adequadamente sumarizada em razão ao seu tipo de dado.

Diante disso, optamos por remover a música que apresentava valor nulo na coluna de streams, tanto por não conter uma informação essencial para o projeto quanto para evitar inconsistência nas análises futuras.

Linha	track_id	track_name	artist_s__name	in_spotify_charts	streams
1	4061483	Love Grows (Where My Rosemary Goes)	Edison Lighthouse	0	BPM110KeyAModeMajorDanceability53Valence75Energy69Acousticness7Instrumentalness0Liveness17Speechiness3

a. Agrupar dados de acordo com variáveis categóricas:

No objetivo de **agrupar dados de acordo com variáveis categóricas**, construímos tabelas-matrizes que agrupavam o total de faixas por artista distribuídas por seus respectivos anos de lançamento. Elaboramos também, uma tabela com informações sobre o total de músicas lançadas por ano, dentro do intervalo temporal presente no dataset.

Neste ponto, também manipulamos esses dados através de gráficos de barra e linha.

---

b. Aplicar medidas de tendência central:

Analisamos o número total de streams por ano. A partir dessa análise, obtivemos insights muito relevantes, como a concentração do pico de streams em músicas lançadas a partir do ano de 2017.

É importante destacar que, embora tenhamos utilizado gráficos de linha, que possuem a essência de demonstrar tendências temporais, neste projeto, a ilustração baseada nos anos serve apenas como referência informativa em relação ao ano de lançamento. Isso ocorre porque a base de dados contempla informações de músicas mais populares no ranking do Spotify em 2023. Ou seja, independente do ano de lançamento, todos os dados relacionados a fatores de sucesso, correspondem ao ano de 2023.

Dessa forma, aplicamos as medidas de tendência central em variáveis numéricas, com o objetivo de obter uma compreensão inicial e objetiva das relações que posteriormente seriam aprofundadas com testes mais avançados para responder às hipóteses do projeto. Além disso, a análise permitiu identificar os outliers e observar como esses valores discrepantes se comportavam com as demais variáveis.

A partir dessas análises, obtivemos insights relevantes, como o ano com maior acúmulo de streams, o nível de BPM associado à maior média de streams e o artista que possuía a maior quantidade total de streams.

Paralelamente, também foram elaborados gráficos do tipo histograma, com o auxílio da linguagem Python, para examinar a distribuição dos dados, como por exemplo: a quantidade de músicas em cada faixa de BPM.

---

c. Aplicar medidas de dispersão:

Por meio de uma tabela-matriz, sumarizamos dados relacionados a fatores de sucesso das músicas, como posição dos charts em cada plataforma analisada e a média de playlists nas quais essas faixas estavam inseridas. Além disso, calculamos o desvio padrão para avaliar o quão dispersos esses valores estavam em relação à média.

Os resultados revelam uma discrepância considerável: as variáveis apresentaram desvios padrão significativamente altos, indicando que os valores individuais se afastam bastante da média. Um exemplo marcante foi a variável *in_deezer_playlists*, cuja média foi de 388 playlists, enquanto o desvio padrão ultrapassou três vezes esse valor, atingindo 1.132.

Esse resultado foi problemático porque indicou uma grande variabilidade nos dados, dificultando a identificação de padrões consistentes. além disso, uma dispersão tão elevada compromete a estabilidade de análises estatísticas mais avançadas, como a regressão, que pressupõem uma variabilidade mais controlada para gerar modelos preditivos mais confiáveis.

Apesar dessas análises iniciais indicarem uma alta dispersão e, consequentemente, um maior risco na interpretação estatística, optamos por manter todos os dados no estudo. essa decisão se justifica pelo contexto específico do setor musical que naturalmente apresenta números mais expressivos, como músicas inseridas em milhares de playlists ou com volumes de streams extremamente altos. Assim, consideramos que esses valores refletem adequadamente a realidade do domínio analisado.

No entanto, adotamos cuidados adicionais, especialmente em relação às variáveis numéricas, garantindo que valores incoerentes – como números inexplicavelmente altos, fora de um intervalo numérico considerável, ou negativos, que não fariam sentido no contexto analisado – fossem identificados e não passassem despercebidos.

d. Calcular os quartis

Subsequentemente, aplicamos a técnica de segmentação por meio do **cálculos dos quartis**. Inicialmente, dividimos o dataset em quatro partes iguais com base na coluna streams, que neste caso foi considerada a nossa principal variável indicadora de sucesso. O objetivo dessa segmentação foi classificar cada uma dessas faixas em categorias: Muito Baixo, ‘Baixo’, ‘Médio’ e ‘Alto’, permitindo identificar quais músicas apresentavam um volume elevado de streams e quais registravam valores muito baixos.

Além disso, adicionamos operações básicas à consulta para expor os intervalos correspondentes a cada quartil, ou seja, identificar explicitamente quais valores de streams cada segmento abrangia.

```
WITH quartil_streams AS (
  SELECT
    track_id,
```

```
    CAST(streams AS INT64) AS streams,
    NTILE(4) OVER (ORDER BY CAST(streams AS INT64)) AS quartil_
streams
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
  WHERE SAFE_CAST(streams AS INT64) IS NOT NULL
)
SELECT
  quartil_streams,
  MIN(streams) AS min_streams,
  MAX(streams) AS max_streams,
  COUNT(*) AS qtd_musicas
FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
GROUP BY quartil_streams
ORDER BY quartil_streams;
```

Linha	quartil_streams	min_streams	max_streams	qtd_musicas
1	1	2762	141381703	237
2	2	141720999	288101651	237
3	3	290228626	672656250	237
4	4	673801126	3703895074	237

e. Calcular as correlações entre variaveis

Neste contexto, é importante destacar que, para as variáveis categóricas *mode* e *key*, ambas características musicais, criamos versões numéricas com o objetivo de realizar testes iniciais de correlação com a variável streams. Como essas variáveis são originalmente categóricas, métodos estatísticos que exigem variáveis numéricas não poderiam ser diretamente aplicados.

Entretanto, na aplicação dos testes estatísticos mais avançados, optamos por manter essas variáveis em seu formato original, preservando sua natureza categórica para garantir a validade e coerência das análises. Mais adiante, explicaremos com detalhes esses procedimentos.

A correlação entre as variáveis que pertenciam as hipóteses 1, 3 e 4, foram calculadas em uma única query, o resultado que obtivemos foram os seguinte:

```
---hipotese 1
WITH correlacao_bpm AS (
```

```
  SELECT CORR(bpm, streams) AS correlacao_bpm
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
),
---hipotese 3
correlacao_playlists_streams AS (
  SELECT CORR(soma_playlists, streams) AS correlacao_playlists_
streams
  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
),
---hipotese 4
correlacao_streams_tracks AS (
  SELECT CORR(total_tracks, total_streams) AS correlacao_stream
s_tracks_sp
  FROM `projeto2lab.projetohipoteses_lab2.artists_streams_spoti
fy`
)
SELECT
cb.correlacao_bpm,
  cps.correlacao_playlists_streams,
  cst.correlacao_streams_tracks_sp
FROM correlacao_bpm AS cb
CROSS JOIN correlacao_playlists_streams AS cps
CROSS JOIN correlacao_streams_tracks AS cst
```

Linha	correlacao_bpm	correlacao_playlists_streams	correlacao_streams_tracks_sp
1	-0.002305066910838731	0.78353745869169067	0.778944198262281

Para a primeira hipótese, obtivemos resultados fracos e negativos, indicando ausência de correlação significativa, já que valores próximos de zero representam relações inexistentes ou muito fracas. Em contrapartida, as demais hipóteses apresentam correlações fortes e positivas, sugerindo uma associação robusta entre as variáveis analisadas.

Para a segunda hipótese, utilizamos as seguintes consultas:

```
---hipotese 2
```

```sql
WITH sp_charts_ap_charts AS (
SELECT
CORR (in_spotify_charts,in_apple_charts) AS correlacao_sp_ap
 FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
),
sp_charts_dz_charts AS (
SELECT
CORR (in_spotify_charts,in_deezer_charts) AS correlacao_sp_dz
 FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
),
sp_charts_sz_charts AS (
SELECT
CORR (in_spotify_charts,in_shazam_charts) AS correlacao_sp_sz
 FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa`
)
SELECT
ap.correlacao_sp_ap,
dz.correlacao_sp_dz,
sz.correlacao_sp_sz,
FROM sp_charts_ap_charts AS ap
CROSS JOIN sp_charts_dz_charts AS dz
CROSS JOIN sp_charts_sz_charts AS sz
```

Linha	correlacao_sp_ap	correlacao_sp_dz	correlacao_sp_sz
1	0.551872726052112	0.59969059415107229	0.60532239239212626

Para essa hipótese, obtivemos uma correlação moderada em relação entre o Spotify e as demais plataformas. Ou seja, indicam uma associação consistente, mas não forte. Isso sugere que, embora cada plataforma tenha seu próprio ritmo de crescimento, há uma relação relevante entre elas e a plataforma do Spotify.

Para a última hipóteses, utilizamos a consulta a seguir:

```sql
--- hipotese 5

WITH tabela_transformada AS (
  SELECT
    streams,
    bpm,
```

```
    `danceability_%`,
    `valence_%`,
    `energy_%`,
    `acousticness_%`,
    `instrumentalness_%`,
    `liveness_%`,
    `speechiness_%`,

    CASE WHEN mode = "Minor" THEN 1 ELSE 0 END AS mode_inter,

    CASE
      WHEN key = "Não Informado" THEN 0
      WHEN key = "C#" THEN 1
      WHEN key = "D" THEN 2
      WHEN key = "D#" THEN 3
      WHEN key = "E" THEN 4
      WHEN key = "F" THEN 5
      WHEN key = "F#" THEN 6
      WHEN key = "G" THEN 7
      WHEN key = "G#" THEN 8
      WHEN key = "A" THEN 9
      WHEN key = "A#" THEN 10
      WHEN key = "B" THEN 11
      ELSE NULL
    END AS key_inter


  FROM `projeto2lab.projetohipoteses_lab2.tabela_completa_limpa
`
  WHERE SAFE_CAST(streams AS INT64) IS NOT NULL
)


SELECT
  CORR(streams, bpm) AS correl_bpm,
  CORR(streams, `danceability_%`) AS correl_danceability,
  CORR(streams, `valence_%`) AS correl_valence,
  CORR(streams, `energy_%`) AS correl_energy,
  CORR(streams, `acousticness_%`) AS correl_acousticness,
```

```
  CORR(streams, `instrumentalness_%`) AS correl_instrumentalnes
s,
  CORR(streams, `liveness_%`) AS correl_liveness,
  CORR(streams, `speechiness_%`) AS correl_speechiness,
  CORR(streams, mode_inter) AS correl_mode,
  CORR(streams, key_inter) AS correl_key
FROM tabela_transformada
```

Obtivemos os seguintes valores como resultado:

Linha	correl_bpm	correl_danceability	correl_valence	correl_energy
1	-0.0023050669108387236	-0.10556527471495256	-0.041632481511413355	-0.0259269801412275

correl_acousticness	correl_instrumentalness	correl_liveness	correl_speechiness
-0.0049995165987254918	-0.044165554429157233	-0.049473459482902368	-0.11275108357318506

correl_mode	correl_key
-0.042394770188565328	-0.049948023400078825

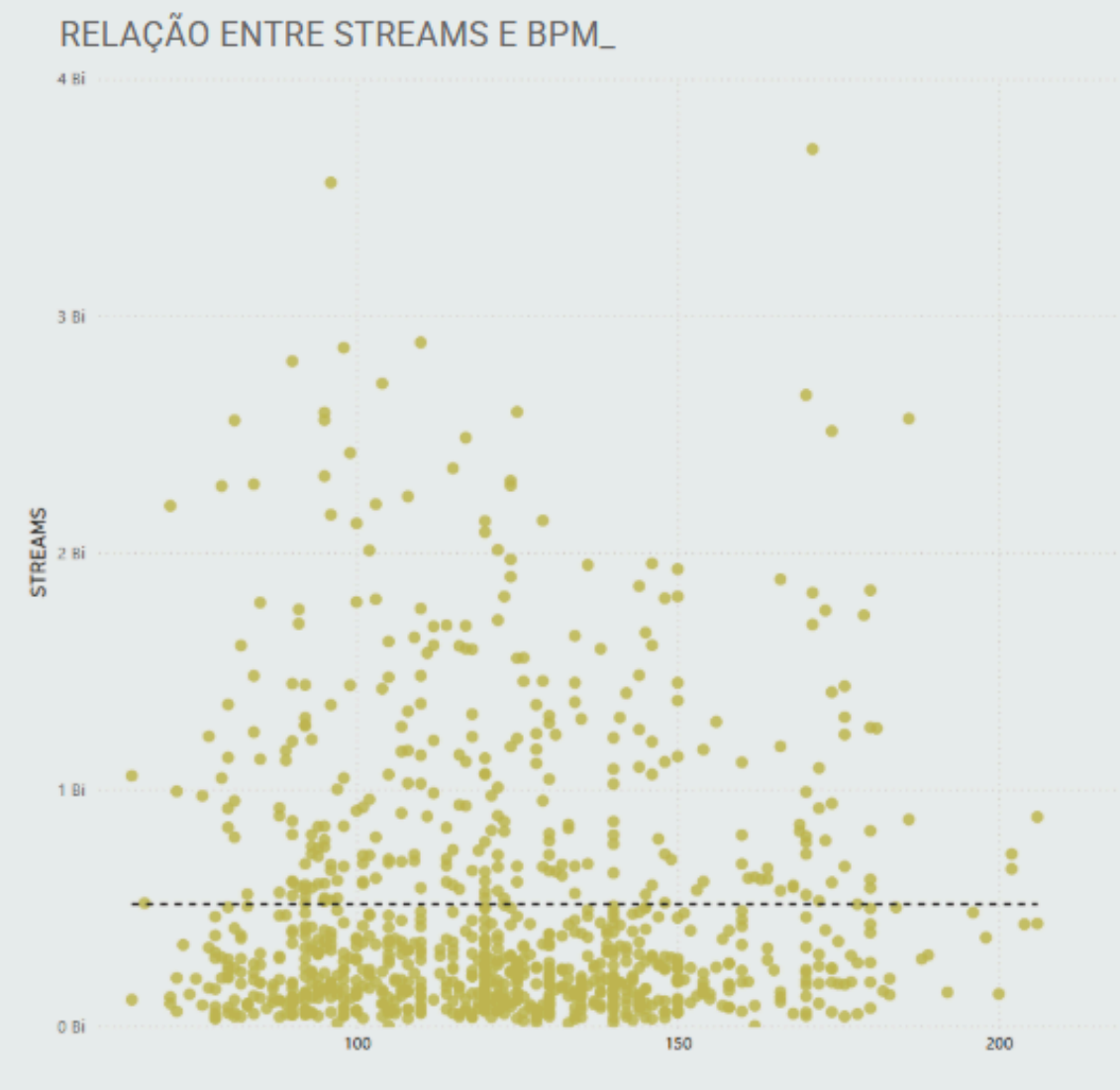
Todas as correlações obtidas apresentam resultados negativos e fracos, indicando que não há uma relação entre as variáveis analisadas. Esse resultado sugere que, quando uma variável aumenta, a outra tende a diminuir, mas essa associação ocorre de forma fraca e sem relevância estatística.

▼ Análises aplicadas

▼ **Hipótese 1** - Músicas com BPM mais altos fazem mais sucesso no Spotify.

Sobre essa hipótese a análise estatística indicou o contrário: **não há relação significativa entre o BPM de uma música e o seu número de streams.**

A correlação entre as duas variáveis foi praticamente nula e negativa ($r \approx -0{,}002$), sugerindo ausência de associação entre o ritmo da música e seu desempenho em popularidade. Essa conclusão se manteve tanto na inspeção visual do gráfico de dispersão quanto nos testes estatísticos mais aprofundados.



Antes de aplicar a regressão linear, realizamos o **teste de normalidade de Shapiro-Wilk**, que indicou que as variáveis BPM e Streams **não seguem uma distribuição normal** (p-valor < 0.001). Esse resultado reforça a presença de **outliers e assimetria** na distribuição dos dados, o que pode impactar negativamente a performance de modelos lineares.

▼ **Teste de Shapiro-Wilk**

```
from scipy.stats import shapiro

stat_bpm, p_bpm = shapiro(df['bpm'])
print(f'BPM | Estatística={stat_bpm:.4f}, p-valor={p_bpm:.4
f}')
```

```
if p_bpm > 0.05:
    print("→ BPM: Distribuição Normal\n")
else:
    print("→ BPM: Não é Normal\n")

stat_streams, p_streams = shapiro(df['streams'])
print(f'Streams | Estatística={stat_streams:.4f}, p-valor={p
_streams:.4f}')
if p_streams > 0.05:
    print("→ Streams: Distribuição Normal\n")
else:
    print("→ Streams: Não é Normal\n")
```

```
BPM | Estatística=0.9775, p-valor=0.0000
→ BPM: Não é Normal

Streams | Estatística=0.7611, p-valor=0.0000
→ Streams: Não é Normal
```

Ainda assim, ajustamos um modelo de regressão linear simples, com BPM como variável independente e streams como variável dependente. O modelo retornou os seguintes resultados:

- **Intercepto ≈ 520 milhões de streams** (estimativa de streams para o menor valor de BPM observado);
- **Coeficiente ≈ −46 mil**: a cada aumento de 1 BPM, a expectativa é de uma **redução de cerca de 46 mil streams**, o que representa um efeito muito pequeno na prática;
- **R^2 ≈ 0.000005**, indicando que o modelo praticamente **não explica a variação** na quantidade de streams.

▼ **Regressão Linear Simples**

```
import matplotlib.pyplot as plt
import seaborn as sns
import pandas as pd
import statsmodels.api as sm

X = df['bpm']
y = df['streams']
```

```
X = sm.add_constant(X)

modelo = sm.OLS(y, X).fit()

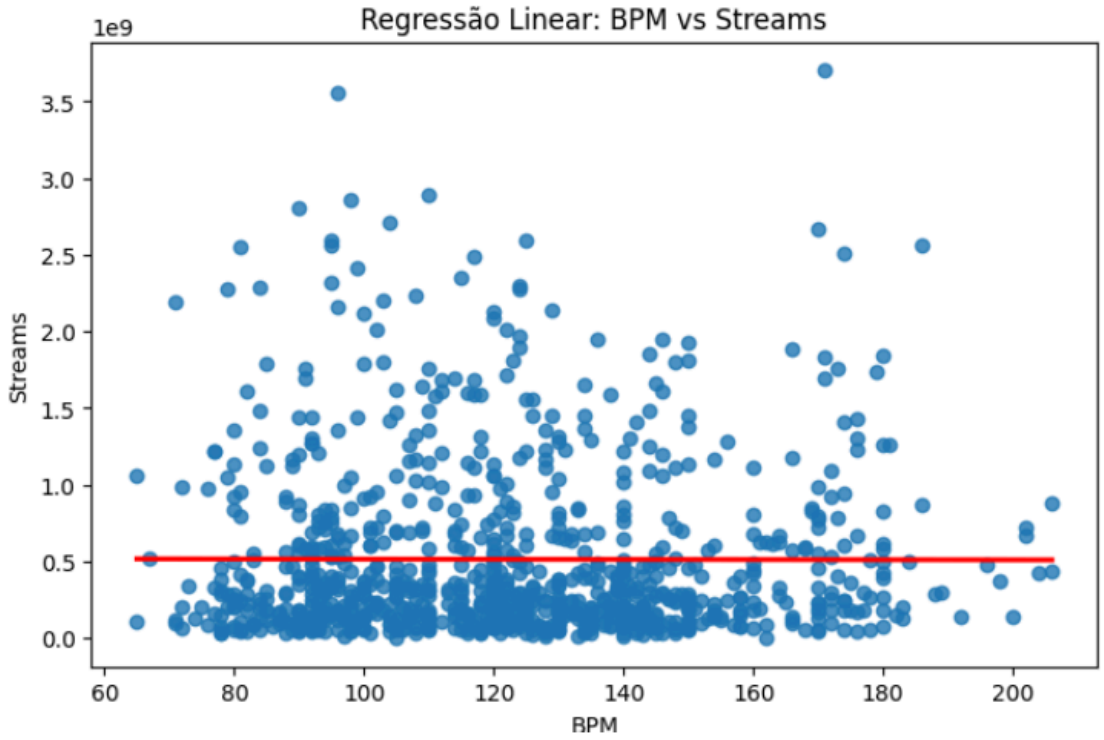
intercepto = modelo.params['const']
coeficiente = modelo.params['bpm']
r_quadrado = modelo.rsquared

bpm_valor = 120
previsao = intercepto + coeficiente * bpm_valor

print(f"Intercepto: {intercepto}")
print(f"Coeficiente: {coeficiente}")
print(f"R²: {r_quadrado}")
print(f"Previsão de streams para BPM={bpm_valor}: {previsa
o}")

plt.figure(figsize=(8,5))
sns.regplot(x='bpm', y='streams', data=df, ci=None, line_kws
={"color":"red"})
plt.title('Regressão Linear: BPM vs Streams')
plt.xlabel('BPM')
plt.ylabel('Streams')
plt.show()
```

```
Intercepto: 520051678.05007744
Coeficiente: -46664.21751409257
R²: 5.313333463541348e-06
Previsão de streams para BPM=120: 514451971.9483863
```



Por exemplo, para uma música com 120 BPM, a previsão do modelo é de aproximadamente **514 milhões de streams**, o que não se difere significativamente da média geral. Com isso, podemos afirmar que a hipótese não se confirma: **o BPM não é um fator relevante para o sucesso de uma música no Spotify**.

▼ **Hipótese 2** - As músicas mais populares no Spotify também possuem um comportamento semelhante em outras plataformas como Deezer, Apple Music e Shazam.

De forma geral, observamos que **sim, existe uma tendência**: músicas bem colocadas no ranking do Spotify **tendem a se destacar** nas demais plataformas. No entanto, **essa relação não é uma regra absoluta**, e há variações.

As **correlações entre as posições nos rankings** reforçam esse comportamento:

- Spotify × Deezer: **r = 0.60**
- Spotify × Apple: **r = 0.55**
- Spotify × Shazam: **r = 0.61**

Esses valores indicam correlações **moderadas e positivas**, sugerindo que o desempenho tende a crescer junto entre as plataformas, mas sem uma correspondência exata.

Para validar essas relações, realizamos o **teste de Shapiro-Wilk** para verificar a normalidade das variáveis, e o resultado foi **p-valor < 0.001 para todas as plataformas**. Isso indica ausência de distribuição normal, com **presença de outliers e assimetria** nas posições, o que novamente exige cautela ao interpretar os modelos lineares.

▼ **Teste de Shapiro-Wilk**

```
import pandas as pd
from scipy import stats

df = pd.read_csv('tabela_completa_limpa.csv')

df['in_shazam_charts'] = df['in_shazam_charts'].fillna(0)

def testar_normalidade(coluna):
    stat, p = stats.shapiro(df[coluna])
    print(f'{coluna} → Estatística={stat:.4f}, p-valor={p:.4
f}')
    if p > 0.05:
        print('→ Distribuição Normal\n')
        return True
    else:
        print('→ Não é Normal\n')
        return False

normal_sp = testar_normalidade('in_spotify_charts')
normal_ap = testar_normalidade('in_apple_charts')
normal_dz = testar_normalidade('in_deezer_charts')
normal_sz = testar_normalidade('in_shazam_charts')
```

```
in_spotify_charts → Estatística=0.6672, p-valor=0.0000
→ Não é Normal

in_apple_charts → Estatística=0.8848, p-valor=0.0000
→ Não é Normal

in_deezer_charts → Estatística=0.5052, p-valor=0.0000
→ Não é Normal

in_shazam_charts → Estatística=0.3978, p-valor=0.0000
→ Não é Normal
```

Mesmo assim, ajustamos **modelos de regressão linear simples** para estimar a posição de uma música em outras plataformas com base em sua posição no Spotify. Os principais resultados foram:

- **Apple Music**: intercepto ≈ **35ª posição;** coeficiente ≈ **+2 posições a cada +1 posição no Spotify,** estabilidade fraca entre as plataformas**;** R^2 ≈ **30%**
- **Deezer**: intercepto ≈ **1ª posição;** coeficiente ≈ **+0.18 posição**, há uma certa estabilidade no desempenho entre as plataformas; R^2 ≈ **36%**
- **Shazam**: intercepto ≈ **1ª posição;** coeficiente ≈ **+5 posições,** estabilidade moderada entre as plataformas**;** R^2 ≈ **37%**

Ou seja, se uma música estiver na **1ª posição no Spotify**, as previsões médias seriam:

- Apple Music: **36ª posição**
- Deezer: **1ª posição**
- Shazam: **4ª posição**

▼ **Regressão Linear Simples**

```
# O mesmo códgio foi utilizado para as três plataformas, ape
nas mudamos os campos correspondentes a cada variável.

import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score
import numpy as np

X = df[['in_spotify_charts']].values
Y = df['in_deezer_charts'].values  # mudamos conforme variáv
```

```
el analisada

modelo = LinearRegression()
modelo.fit(X, Y)

intercepto = modelo.intercept_
coeficiente = modelo.coef_[0]

Y_pred = modelo.predict(X)

r2 = r2_score(Y, Y_pred)

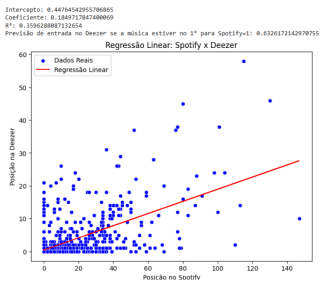
spotify_valor = 1
previsao = modelo.predict([[spotify_valor]])

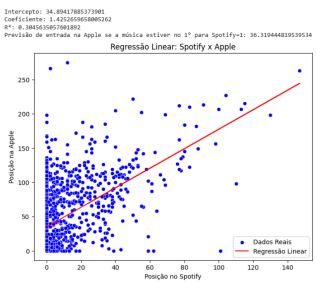
print(f'Intercepto: {intercepto}')
print(f'Coeficiente: {coeficiente}')
print(f'R²: {r2}')
print(f'Previsão de entrada no Deezer se a música estiver no
1° para Spotify={spotify_valor}: {previsao[0]}') # mudamos c
onforme variável analisada

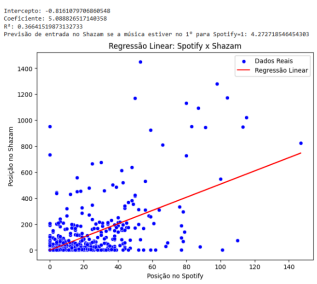
plt.figure(figsize=(8,6))
sns.scatterplot(x=X.flatten(), y=Y, color='blue', label='Dad
os Reais')
plt.plot(X, Y_pred, color='red', label='Regressão Linear')

plt.xlabel('Posição no Spotify')
plt.ylabel('Posição na Deezer')  # mudamos conforme variável
analisada
plt.title('Regressão Linear: Spotify x Deezer')  # mudamos c
onforme variável analisada
plt.legend()

plt.show()
```







Com base nisso, podemos afirmar que **a hipótese é parcialmente confirmada**. Existe uma **tendência moderada de sincronia entre as plataformas**, mas ela **não é determinante**. O sucesso no Spotify pode ajudar a impulsionar uma música em outras plataformas, mas não é suficiente por si só.

▼ **Hipótese 3** - A presença de uma música em um maior número de playlists está relacionada a um maior número de streams.

Nesta análise, buscamos compreender se a inclusão de uma música em playlists impacta diretamente no seu volume de streams. Os resultados apontam uma **relação forte e positiva** entre essas duas variáveis: **quanto mais playlists uma música integra, maior tende a ser o seu número de reproduções**.

A correlação observada foi de **r = 0.78**, indicando uma associação consistente. No entanto, é importante destacar que **correlação não implica causalidade**. Ou seja, **não é possível afirmar com certeza se estar em mais playlists causa mais streams, ou se músicas que já são muito populares acabam sendo mais adicionadas às playlists**. A relação é forte, mas a direção do impacto não pode ser confirmada apenas com esse modelo.

O **teste de Shapiro-Wilk** indicou que ambas as variáveis — *streams* e *número de playlists* — **não seguem distribuição normal** (p-valor < 0.001), sugerindo a presença de **outliers e assimetria**.

▼ **Teste de Shapiro-Wilk**

```
import pandas as pd
from scipy import stats

def testar_normalidade(coluna):
    stat, p = stats.shapiro(df[coluna])
    print(f'{coluna} → Estatística={stat:.4f}, p-valor={p:.4
f}')
    if p > 0.05:
```

```
        print('→ Distribuição Normal\n')
        return True
    else:
        print('→ Não é Normal\n')
        return False

normal_streams = testar_normalidade('streams')
normal_playlists = testar_normalidade('soma_playlists')
```

```
streams → Estatística=0.7611, p-valor=0.0000
→ Não é Normal

soma_playlists → Estatística=0.6056, p-valor=0.0000
→ Não é Normal
```

Mesmo assim, ajustamos um **modelo de regressão linear simples**, que trouxe os seguintes resultados:

- **Intercepto ≈ 231 milhões de streams**: estimativa de reprodução para uma música não presente em nenhuma playlist.
- **Coeficiente ≈ +49 mil**: a cada nova playlist, uma música tende a ganhar, em média, 49 mil streams a mais.
- **R^2 ≈ 61%**: o modelo explica bem a variação nos streams com base na quantidade de playlists.

▼ **Regressão Linear Simples**

```
from sklearn.linear_model import LinearRegression
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

X = df[['soma_playlists']]
y = df['streams']

modelo = LinearRegression()
modelo.fit(X, y)

print(f'Intercepto: {modelo.intercept_}')
```

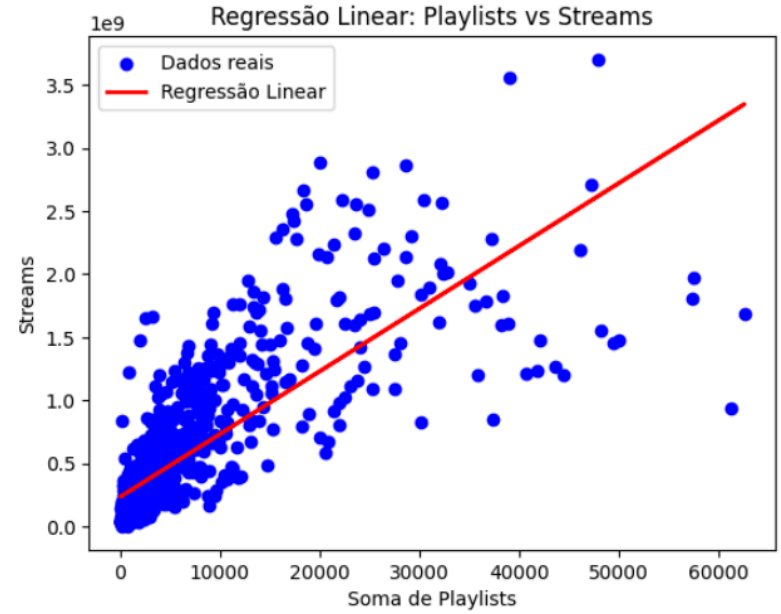
```
print(f'Coeficiente: {modelo.coef_[0]}')

previsao = modelo.predict(pd.DataFrame({'soma_playlists': [5
0]}))
print(f'Previsão de streams para 50 playlists: {previsao
[0]}')

r2 = modelo.score(X, y)
print(f'R²: {r2}')

plt.scatter(X, y, color='blue', label='Dados reais')
plt.plot(X, modelo.predict(X), color='red', linewidth=2, lab
el='Regressão Linear')
plt.xlabel('Soma de Playlists')
plt.ylabel('Streams')
plt.title('Regressão Linear: Playlists vs Streams')
plt.legend()
plt.show()
```

```
Intercepto: 231899956.93222493
Coeficiente: 49795.44017612264
Previsão de streams para 50 playlists: 234389728.94103107
R²: 0.613930949173032
```



Por exemplo, para uma música presente em **50 playlists**, a projeção é de aproximadamente **234 milhões de streams**.

Em resumo, a hipótese é confirmada em termos de associação, mas com a ressalva de que **a relação identificada é correlacional, não causal**. Os dados indicam uma forte tendência, mas não nos permitem afirmar com certeza que as playlists são o fator gerador do sucesso — elas podem ser também consequência dele.

▼ **Hipótese 4** - Artistas com maior número de músicas no Spotify têm mais streams.

Os resultados mostraram uma **correlação forte e positiva** (r ≈ 0.78), sugerindo que artistas com mais faixas geralmente são mais ouvidos. No entanto, essa relação precisa ser melhor interpretada.

Ao observar o **gráfico de dispersão**, percebemos que embora a tendência geral (linha de regressão) aponte para cima — indicando crescimento dos streams conforme o número de músicas aumenta —, há uma **concentração massiva de pontos entre 1 e 5 faixas**. Apenas alguns poucos artistas possuem mais de 20 músicas, o que caracteriza **outliers que influenciam fortemente o valor da correlação**.

Além disso, vale ressaltar um possível **viés de seleção**: a base de dados utilizada contempla apenas as músicas mais populares do Spotify em 2023. Ou seja, artistas com poucos lançamentos e desempenho baixo não estão representados, o que pode distorcer a percepção geral da relação entre quantidade de músicas e popularidade.

Apesar dessas limitações, os testes estatísticos reforçaram a existência de uma relação clara:

- **Teste de Shapiro-Wilk**: tanto o número de músicas quanto os streams apresentaram **p-valor < 0.001**, indicando ausência de distribuição normal e presença de **assimetria e outliers**.

▼ **Teste de Shapiro-Wilk**

```
import pandas as pd
from scipy import stats

df_artistas = df_artistas.fillna(0)

def testar_normalidade(coluna):
    stat, p = stats.shapiro(df_artistas[coluna])
    print(f'{coluna} → Estatística={stat:.4f}, p-valor={p:.4
f}')
```

```
    if p > 0.05:
        print('→ Distribuição Normal\n')
        return True
    else:
        print('→ Não é Normal\n')
        return False

normal_tt = testar_normalidade('total_tracks')
normal_ts = testar_normalidade('total_streams')
```

```
total_tracks → Estatística=0.2144, p-valor=0.0000
→ Não é Normal

total_streams → Estatística=0.4539, p-valor=0.0000
→ Não é Normal
```

- **Regressão linear**: o modelo revelou um **intercepto ≈ 18,9 milhões de streams** (valor teórico para artistas com zero músicas, sem aplicação prática, mas coerente dentro do modelo).
- **Coeficiente ≈ 501 milhões de streams**: para cada nova música adicionada ao catálogo, o artista tende a ganhar, em média, mais 501 milhões de streams.
- **R^2 ≈ 61%**: o modelo tem bom poder explicativo, mostrando que **61% da variação nos streams pode ser explicada pela quantidade de músicas**.

▼ **Regressão Linear Simples**

```
from sklearn.linear_model import LinearRegression
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

x = df[['total_tracks']]
y = df['total_streams']

modelo = LinearRegression ()
modelo.fit(x,y)

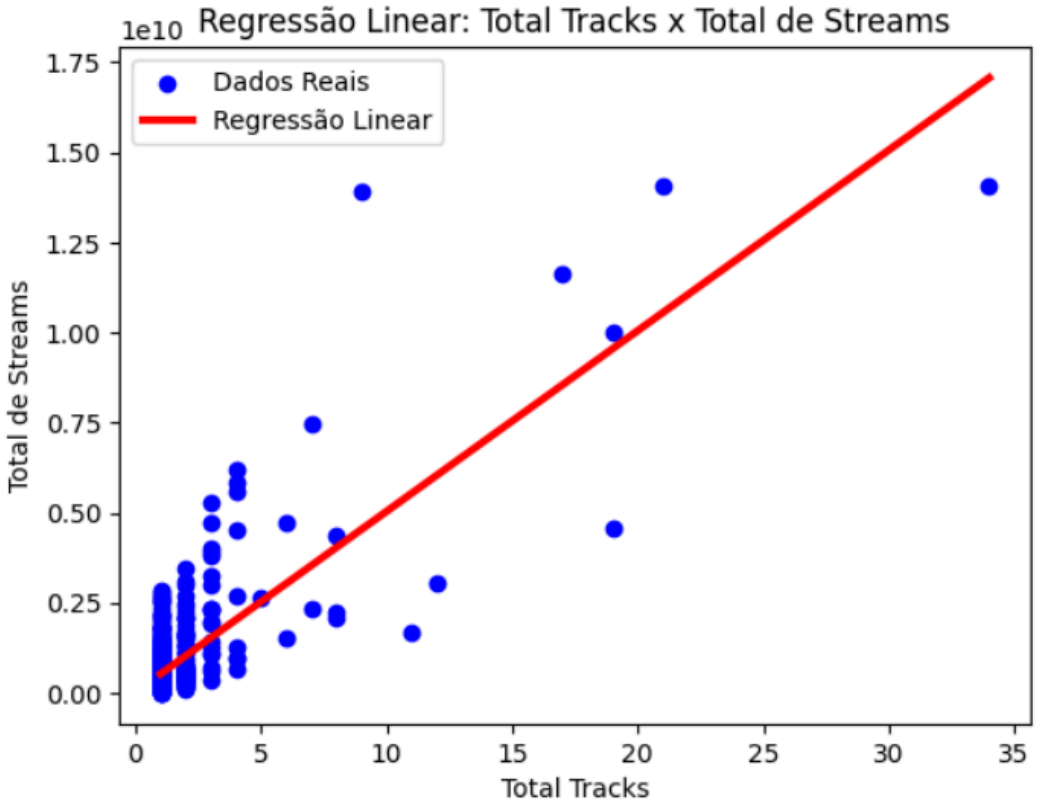
print(f'Intercepto: {modelo.intercept_}')
print(f'Coeficiente: {modelo.coef_[0]}')
```

```
previsao = modelo.predict(pd.DataFrame({'total_tracks':[1
0]}))
print(f'Previsão de Streams para um total de 10 músicas por
artista: {previsao[0]}')

r2 = modelo.score(x,y)
print(f'R²: {r2}')

plt.scatter(x, y, color='blue', label='Dados Reais')
plt.plot(x, modelo.predict(x), color='red', linewidth=3, lab
el='Regressão Linear')
plt.xlabel('Total Tracks')
plt.ylabel('Total de Streams')
plt.title('Regressão Linear: Total Tracks x Total de Stream
s')
plt.legend()
plt.show()
```

```
Intercepto: 18919973.87271893
Coeficiente: 501483732.7457479
Previsão de Streams para um total de 10 músicas por artista: 5033757301.330198
R²: 0.606754064006468
```



Por exemplo, um artista com **10 músicas** pode alcançar, de acordo com a previsão do modelo, aproximadamente **5 bilhões de streams**.

Em resumo, essa hipótese é **confirmada**: artistas com mais músicas disponíveis no Spotify tendem a acumular um número maior de reproduções. Contudo, é importante considerar as **limitações da base de dados** e a **influência de outliers**, que podem intensificar artificialmente a relação observada.

▼ **Hipótese 5** - As características da música influenciam no sucesso em termos de streams no Spotify.

As **correlações encontradas entre as características musicais e os streams foram todas fracas e negativas**, o que indica que não há um padrão claro de que essas qualidades impactem diretamente o sucesso de uma música. Por exemplo:

- **Speechiness** (grau de “fala” na música): r ≈ –0.11
- **Danceability**: r ≈ –0.10

- Demais variáveis (como bpm, key, mode, energy, valence, liveness) variaram entre −0.002 e −0.05

Esses valores sugerem que, **aumentos em atributos como "dançabilidade" ou "fala" podem estar levemente associados a menos streams**, mas o efeito é tão pequeno que não pode ser considerado relevante isoladamente.

Analisamos também as variáveis categóricas por meio de testes específicos para os conjuntos:

- **Tonalidade (mode)** — maior ou menor
- **Nota principal (key)** — de A a G#

Nenhuma delas apresentou diferença significativa quanto ao desempenho em streams.

▼ **Teste de Mann-Whitney (mode)**

```
from scipy.stats import mannwhitneyu

minor = df[df['mode'] == 'Minor']['streams']
major = df[df['mode'] == 'Major']['streams']

stat, p = mannwhitneyu(minor, major)

print('Teste de Mann-Whitney para Mode (Minor x Major)')
print(f'Estatística: {stat:.4f}, p-valor: {p:.4f}')

if p < 0.05:
    print('Diferença significativa entre os grupos.')
else:
    print('Não há diferença significativa entre os grupos.')
```

```
Teste de Mann-Whitney para Mode (Minor x Major)
Estatística: 102993.5000, p-valor: 0.1088
Não há diferença significativa entre os grupos.
```

▼ **Teste de Kruskal-Wallis (key)**

```
from scipy.stats import kruskal

grupos_key = [df[df['key'] == k]['streams'] for k in df['ke
```

```
y'].unique()]

stat, p = kruskal(*grupos_key)

print('Teste de Kruskal-Wallis para Key')
print(f'Estatística: {stat:.4f}, p-valor: {p:.4f}')

if p < 0.05:
    print('Diferença significativa entre as categorias de Ke
y.')
else:
    print('Não há diferença significativa entre as categoria
s de Key.')
```

```
Teste de Kruskal-Wallis para Key
Estatística: 5.2303, p-valor: 0.9195
Não há diferença significativa entre as categorias de Key.
```

TESTE DE SHAPIRO (variáveis numéricas):

Teste de Shapiro-Wilk para **speechiness_%**:
Estatística=0.7151, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **bpm**:
Estatística=0.9775, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **danceability_%**:
Estatística=0.9796, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **valence_%**:
Estatística=0.9773, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **energy_%**:
Estatística=0.9806, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **acousticness_%**:
Estatística=0.8725, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **instrumentalness_%**:
Estatística=0.1865, p-valor=0.0000
Distribuição NÃO parece normal.

Teste de Shapiro-Wilk para **liveness_%**:
Estatística=0.7696, p-valor=0.0000
Distribuição NÃO parece normal.

▼ **Teste de Shapiro-Wilk (variáveis numéricas)**

```
from scipy.stats import shapiro

variaveis_numericas = ['streams', 'bpm', 'danceability_%',
'valence_%', 'energy_%',
                       'acousticness_%', 'instrumentalness
_%', 'liveness_%', 'speechiness_%']

for var in variaveis_numericas:
    stat, p = shapiro(df[var])
    print(f'Teste de Shapiro-Wilk para {var}:')
    print(f'Estatística={stat:.4f}, p-valor={p:.4f}')
    if p > 0.05:
        print('Distribuição parece NORMAL.\n')
    else:
        print('Distribuição NÃO parece normal.\n')
```

Em seguida, aplicamos uma **regressão linear múltipla**, considerando todas as variáveis numéricas técnicas da música simultaneamente. O resultado mostrou que:

- **$R^2 \approx 0.03$**, ou seja, o modelo explica **apenas 3% da variação no número de streams**
- Algumas características continuam apresentando impacto levemente negativo:
  - Músicas mais “faladas” (speechiness) e mais “dançantes” (danceability) tendem a ter um desempenho levemente inferior
- As demais variáveis tiveram efeitos fracos ou estatisticamente insignificantes

▼ **Regressão Linear Múltipla (varáveis numéricas)**

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_in
flation_factor
import numpy as np

X = df[['bpm', 'danceability_%', 'valence_%', 'energy_%',
        'acousticness_%', 'instrumentalness_%', 'liveness
_%', 'speechiness_%']]

y = df['streams']

X_const = sm.add_constant(X)

modelo = sm.OLS(y, X_const).fit()

print(modelo.summary())

vif_data = pd.DataFrame()
vif_data["variável"] = X.columns
vif_data["VIF"] = [variance_inflation_factor(X.values, i) fo
r i in range(X.shape[1])]

print("\nVIFs:")
print(vif_data)
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                streams   R-squared:                       0.029
Model:                            OLS   Adj. R-squared:                  0.021
Method:                 Least Squares   F-statistic:                     3.560
Date:                Mon, 26 May 2025   Prob (F-statistic):           0.000447
Time:                        19:02:43   Log-Likelihood:                -20440.
No. Observations:                 948   AIC:                         4.090e+04
Df Residuals:                     939   BIC:                         4.094e+04
Df Model:                           8
Covariance Type:            nonrobust
==========================================================================================
                             coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------------------
const                  1.037e+09   1.69e+08      6.120      0.000    7.05e+08    1.37e+09
bpm                   -3.041e+05   6.65e+05     -0.457      0.648   -1.61e+06       1e+06
danceability_%        -4.219e+06   1.47e+06     -2.874      0.004    -7.1e+06   -1.34e+06
valence_%              1.928e+05   9.35e+05      0.206      0.837   -1.64e+06    2.03e+06
energy_%              -1.103e+06   1.47e+06     -0.748      0.454      -4e+06    1.79e+06
acousticness_%        -1.127e+06   8.98e+05     -1.255      0.210   -2.89e+06    6.35e+05
instrumentalness_% -4.253e+06    2.2e+06     -1.931      0.054   -8.58e+06    6.89e+04
liveness_%            -2.562e+06   1.35e+06     -1.897      0.058   -5.21e+06    8.86e+04
speechiness_%         -5.735e+06   1.88e+06     -3.046      0.002   -9.43e+06   -2.04e+06
==============================================================================
Omnibus:                      375.911   Durbin-Watson:                   1.691
Prob(Omnibus):                  0.000   Jarque-Bera (JB):             1322.575
Skew:                           1.942   Prob(JB):                    6.40e-288
Kurtosis:                       7.288   Cond. No.                     1.56e+03
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
[2] The condition number is large, 1.56e+03. This might indicate that there are
strong multicollinearity or other numerical problems.

VIFs:
             variável        VIF
0                 bpm  14.150253
1      danceability_%  18.424190
2           valence_%   7.851388
3            energy_%  19.475157
4      acousticness_%   2.349664
5  instrumentalness_%   1.062556
6          liveness_%   2.769069
7       speechiness_%   2.147168
```

Durante essa etapa, identificamos a presença de **multicolinearidade**, ou seja, algumas variáveis são muito parecidas entre si quem podem confundir o modelo (como energy e valence, por exemplo). Apesar disso, optamos por **manter todas as variáveis no estudo com fins didáticos**, para compreender melhor seu comportamento individual.

Em resumo, essa hipótese foi **refutada**: as características técnicas da música **não são determinantes para o sucesso em termos de streams no Spotify**. O sucesso

parece estar muito mais relacionado a fatores externos, como **estratégias de divulgação, inserção em playlists, apoio dos fãs e visibilidade nas mídias sociais**.

▼ Análise de Nicho: TopHits

Após a aplicação da segmentação e a validação das hipóteses levantadas inicialmente, optamos por explorar de forma mais aprofundada um grupo específico da base: as músicas com mais de 1 bilhão de streams, que chamamos de Top Hits. O objetivo foi fornecer à gravadora uma representação clara das principais características musicais associadas às faixas de maior sucesso. A proposta não é afirmar causalidades, mas sim apresentar evidências que contribuam para tomadas de decisão estratégicas, baseadas em padrões extraídos dos dados.

Para isolar esse subconjunto, foi realizada a seguinte consulta SQL:

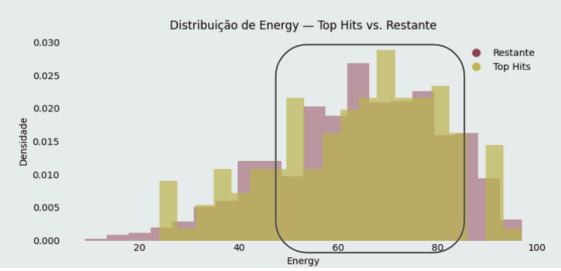
```
SELECT
  track_id,
  track_name,
  artist_name,
  streams,
  artist_count,
  data_lancamento,
  bpm,
  key,
  mode,
  danceability,
  valence,
  energy,
  acousticness,
  instrumentalness,
  liveness,
  speechiness,
  in_spotify_playlists,
  in_apple_playlists,
  in_deezer_playlists,
  in_spotify_charts,
  in_apple_charts,
  in_deezer_charts,
  in_shazam_charts
FROM
```

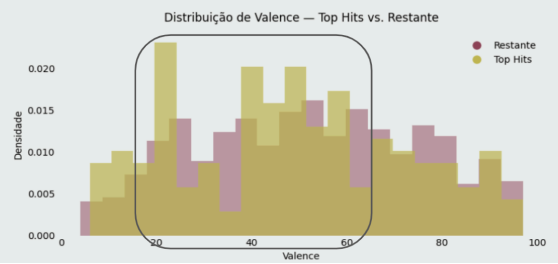
```
  `projeto2lab-457400.projetohipoteses_rv02.limpos_agrupados`
WHERE
  streams >= 1000000000
```

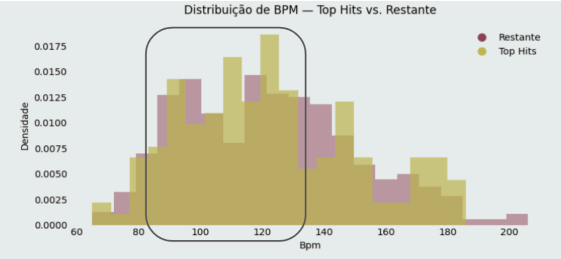
Essa filtragem resultou em um conjunto de 152 músicas, pertencentes a 113 artistas distintos.

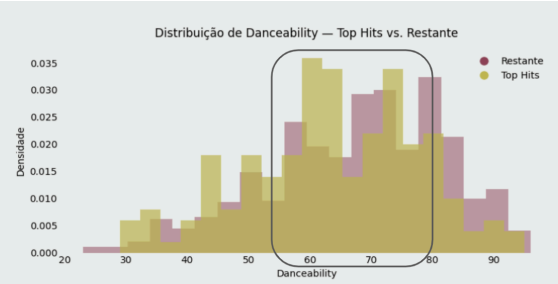
Com essa nova visualização chamada **top_hits**, importada no Power BI, iniciamos comparações entre esse subconjunto e a base completa. Utilizamos histogramas para observar como cada característica musical se comporta dentro desse grupo, como por exemplo o BPM, onde faixas entre 10 e 20 batidas por minuto aparecem com mais frequência. A análise foi aplicada a **10 variáveis**, sendo **8 numéricas** (bpm, danceability, valence, energy, acousticness, instrumentalness, liveness, speechiness) e **2 categóricas** (key e mode).

MÉTRICAS DE RITMO E ENERGIA:

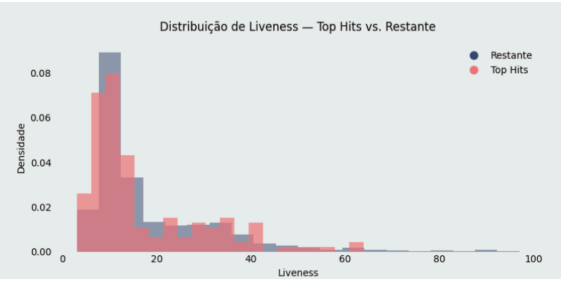


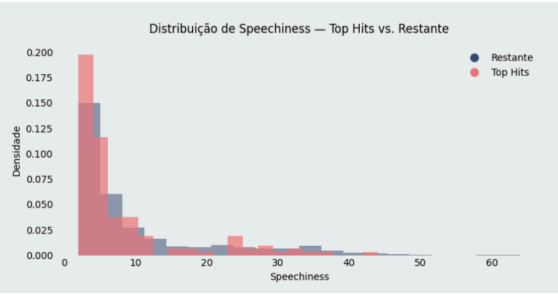


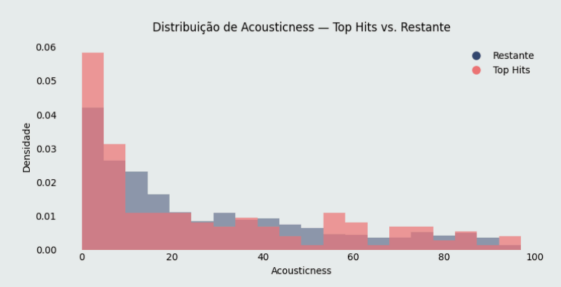


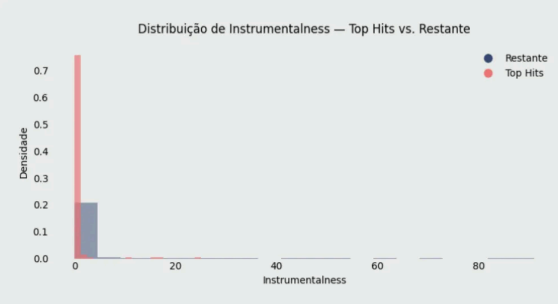


MÉTRICAS DE VOCAL E AMBIÊNCIA

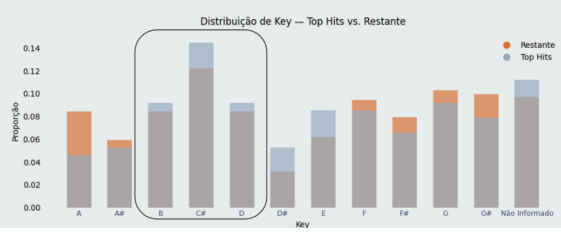


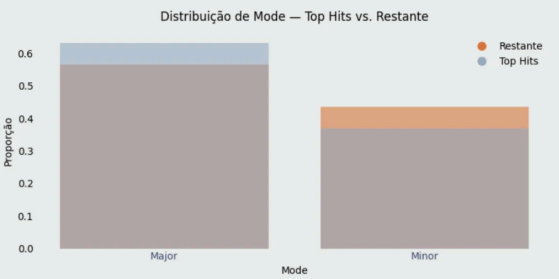






MÉTRICAS DE TONALIDADE



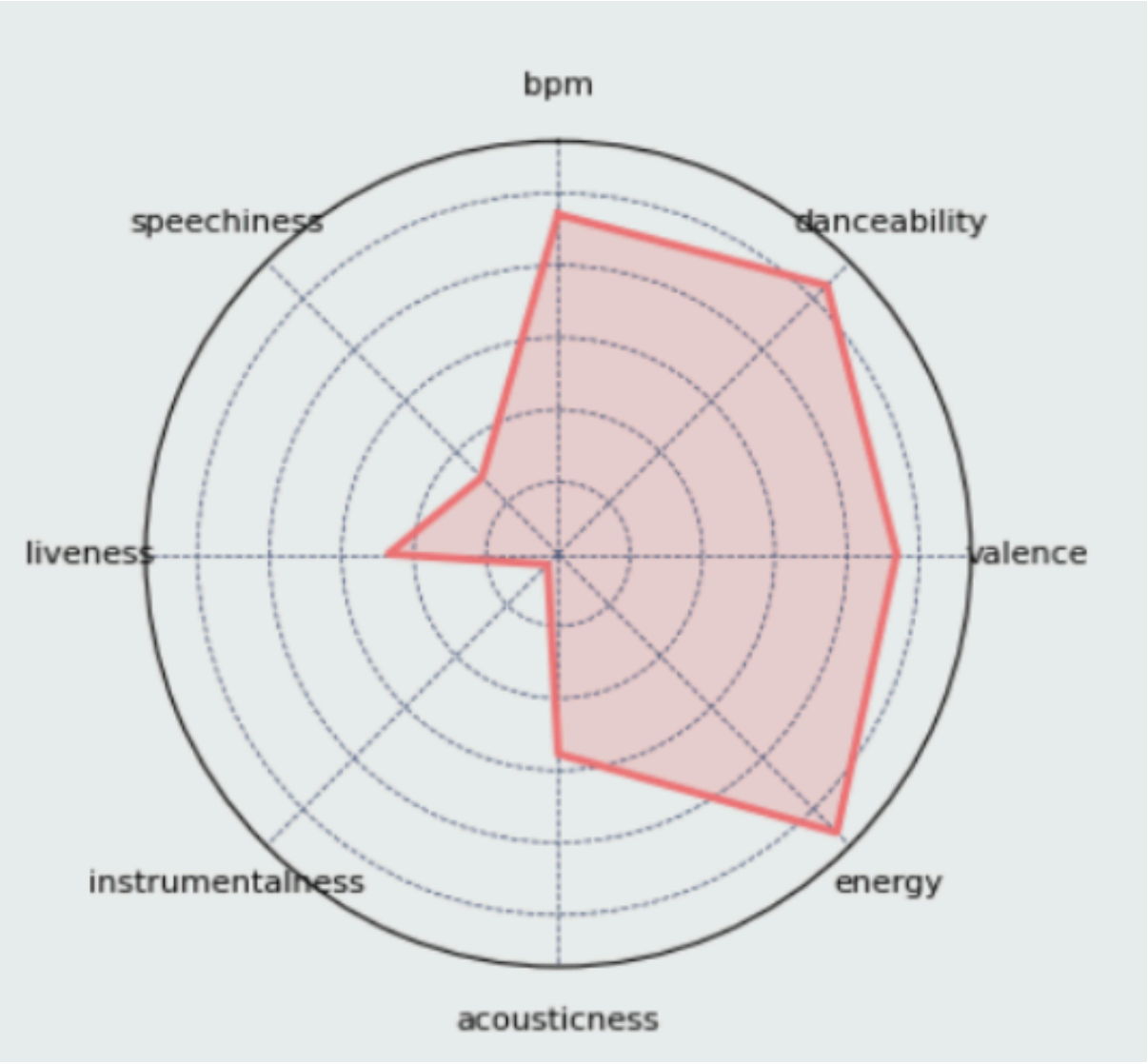


Com base nos dados levantados, elaboramos uma tabela que sintetiza o **padrão médio das faixas Top Hits**, acompanhada de uma interpretação prática dessas informações.

Variável	Padrão dos Top Hits	Interpretação prática
Energy	Pico em 60–80; restante em 80–90;	Hits são enérgicos, porém não os mais “pesados” de todos.
Valence	Pico em 20–30; restante em 40–70;	Hits mantêm “emoção média”; não felicidade extrema.
BPM	Top Hits se concentram em 100–120 bpm;	Ritmos moderados maximizam o apelo global.
Danceability	Pico em ~55–70; restante puxa mais para valores mais altos;	Hits têm batida marcante.
Liveness	Ambos em 5–15, mas restante decai mais rápido após 20;	Sucesso tende ao estúdio; performance “ao vivo” é mais rara.
Speechiness	Top Hits em 1–10;	Hits quase sempre são cantadas (não spoken-word).
Acousticness	Top Hits muito concentrados em 0–10; restante mais espalhado;	Poucos instrumentos “limpos”, muita produção musical.
Instrumentalness	Top Hits quase 100 % em zero.	Raramente são instrumentais.
Mode	Major 63 % Minor 37%	Major é mais frequente.

Variável	Padrão dos Top Hits	Interpretação prática
Key	**C# 14 %**, B 9 %, G# 11 % (maiores)	-

Por fim, para uma visualização mais intuitiva e comparativa, desenvolvemos um **gráfico de radar**, onde é possível observar, de forma resumida, como os Top Hits se posicionam em cada característica musical em uma escala de 0.1 (baixo) a 0.5 (alto). Esse gráfico busca traduzir, em uma imagem, o perfil médio das músicas que alcançaram maior número de streams, oferecendo à gravadora um recurso de leitura rápida para identificar possíveis padrões de sucesso.



Levando em consideração tudo o que analisamos e os gráficos apresentados, nota-se que, nas oito características numéricas estudadas, os Top Hits de 2023 posicionam-se sempre numa faixa intermediária – nem “superdançantes”, nem “baladas lentas” –, com

valores moderados-altos de danceability, BPM, energy e valence. Em contrapartida, apresentam liveness, speechiness, acousticness e instrumentalness muito baixos, o que indica faixas predominantemente gravadas em estúdio, com forte presença de vocais e bastante produção. Já na dimensão categórica, vemos que 63 % desses hits estão em modo Major e que tonalidades “quentes” como C♯, G♯ e B aparecem com frequência acima da média, dando às músicas um caráter mais vibrante, radiante e expansivo. Em suma, o perfil médio dos sucessos bilionários de 2023 é mid-upbeat: energéticos mas sem extremos, focados em vocais e em tonalidades luminosas.

▼ Resultados e Conclusões.

▼ Recomendações:

Com base nas análises realizadas e nas limitações observadas ao longo do processo, elaboramos uma série de recomendações que podem aprofundar os insights e ajudar futuras investigações. Esses próximos passos buscam ampliar o escopo dos dados e explorar relações que ficaram apenas sugeridas nesta primeira análise.

1. Corrigir e explorar a temporalidade

Durante o processo de limpeza de dados, **identificamos várias datas de lançamento inconsistentes**, o que impediu uma análise confiável da evolução dos streams ao longo do tempo. Recomenda-se revisar e padronizar essas datas para permitir estudos temporais.

2. Explorar efeitos por gênero musical

Embora o BPM tenha apresentado correlação nula com os streams de forma geral, **esse padrão pode variar significativamente entre gêneros musicais**. Para investigar isso pode ser adicionada a base a variável de Gênero musical.

3. Ampliar a amostra de músicas

A base utilizada inclui apenas as faixas mais populares de cada artista. Isso pode distorcer os resultados e esconder efeitos relevantes no comportamento geral de consumo. Recomendamos incluir **todas as músicas publicadas**.

▼ Direcionamentos Finais:

🎼***COMPOSIÇÃO MUSICAL***
MID-UPBEAT É O CAMINHO

➡️SOON ***ESTRATÉGIAS DE LANÇAMENTO - INVESTIGAÇÕES***

Foque em BPM 100–120, danceability e energy entre 60–80 (vibrantes, sem exageros).
PRIORIZE O MODO MAJOR
63 % dos “bilionários” estão em Major.
TONALIDADES QUENTES
Experimente C♯, G♯ ou B, as keys com leve sobre-representação nos Top Hits.
VOCAIS EM PRIMEIRO PLANO
Speechiness e instrumentalness quase zero → voz clara ressoa melhor globalmente.

FUTURAS
IMPULSO POR PLAYLISTS (?)
Identifique os tipos de playlists (editorial vs. user-generated) que mais geram streams. *Teste:* compare correlação restringindo a playlists editoriais.
BPM vs STREAMS A PARTIR DE GÊNERO
*P*ode haver efeito de gênero musical. *Teste:* calcule r(bpm, streams) dentro de cada gênero (pop, eletrônico, rock...).
RANKINGS CROSS-PLATAFORMA **A**valie a consistência temporal dos rankings. *Teste:* compare posições Spotify vs. Deezer mês a mês.

▼ Links e arquivos uteis

▼ Arquivo Power BI

proejto2_hipoteses_mec_e_wml.pbix

▼ Link para o GoogleColab

https://colab.research.google.com/drive/1quOxk92ReWGKQInZWJTejArk3STQUP76?usp=sharing

▼ Link da Apresentação

https://docs.google.com/presentation/d/1YaAIAVo6Mgmx3_J5_hPPOMxjTDW1e4HEAwfG4YMLL7s/edit?usp=sharing